

Dimostrare che un linguaggio è regolare sse. esiste un'espressione regolare che lo descrive

$$REG \equiv L(ve) \Leftrightarrow L(ve) \subseteq REG \wedge REG \subseteq L(ve):$$

- $L \in REG \Rightarrow \exists \text{ espr. reg } R \text{ t.c. } L(R) = L$
- $L \in L(ve) \Rightarrow \exists \text{ DFA/NFA } N \text{ t.c. } L(N) = L$

$\Rightarrow$ L'idea è trasformare il DFA/NFA N in un'espressione regolare ve .

Per fare ciò supponiamo che N sia nella sua forma generalizzata G . Abbiamo mostrato a lezione che $\forall \text{NFA } N \exists \text{GNFA } G \text{ t.c. } L(N) = L(G)$.

Forse si può tagliare... il Prof. sa' cos'è un GNFA

Se G è la forma generalizzata di N significa che:

- Le etichette sugli archi sono tutte espr. regolari.
- Lo stato iniziale q_{START} ha solo archi uscenti verso tutti gli altri nodi.
- G ha un solo stato finale q_{acc} con archi entranti da tutti gli altri nodi e nessun arco uscente.
- Per ogni altra coppia di nodi, cioè $\forall (p, q) \in Q \setminus \{q_{acc}\} \times Q \setminus \{q_{START}\}$, inseriamo un arco $p \rightarrow q$ che li collega (eventualmente con etichetta $\emptyset$ se nell'NFA originale non era presente l'arco $p \rightarrow q$).

Una volta trasformato N nel GNFA G equivalente applichiamo l'algoritmo di conversione $Convert(G)$, che trasforma G in un'espr. regolare ve . Tale algoritmo rimuove, ad ogni passo di computazione, uno stato intermedio q_{rip} tra q_{START} e q_{acc} adattando le espr. reg. sugli archi in modo da lasciare il linguaggio invariato. L'obiettivo è quindi arrivare ad avere solo 2 stati: $q_{START} \xrightarrow{ve} q_{acc}$ con $ve \in REGEX$ t.c. $L(ve) = L(G)$.

Convert(G):

- Su input $G = (Q, \Sigma, \delta, q_{\text{START}}, \{q_{\text{ACC}}\})$
- sia $K = |Q|$
 - se $K = 2$ allora G è già nella forma $q_{\text{START}} \xrightarrow{re} q_{\text{ACC}}$. Quindi l'algoritmo termina e ritorna $G' = G$
 - se $K > 2$ scegliamo uno stato $q_{\text{RIP}} \in Q \setminus \{q_{\text{START}}, q_{\text{ACC}}\}$ e costruiamo $G' \in \text{CNFA}$ t.c. $G' = (Q', \Sigma, \delta', q_{\text{START}}, \{q_{\text{ACC}}\})$ in cui

1) $Q' = Q \setminus \{q_{\text{RIP}}\}$

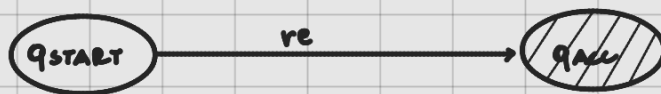
2) $\delta': Q \setminus \{q_{\text{ACC}}\} \times Q \setminus \{q_{\text{RIP}}\} \longrightarrow R_{\epsilon}(\Sigma)$ t.c.

$$\delta'(q_i, q_j) = R_{i,j} \cup (R_{i,\text{RIP}} \cdot (R_{\text{RIP},\text{RIP}})^* \cdot R_{\text{RIP},j})$$

con $q_i, q_j \in Q$ e $R_{i,j} = \delta(q_i, q_j)$

- Output $\langle G' \rangle$

Dopo aver eseguito Convert(G) avremo un G' :



A questo punto è evidente che tutte le stringhe accettate da G sono solo quelle che rispettano la espr. reg. re . Dunque $L(G) = L(N) = L(re)$

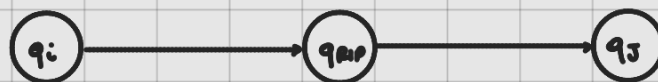
Rimane solo da dimostrare che Convert(G) è equivalente a G . Procediamo per induzione

HP INDUTTIVA: G con $K-1$ stati è equivalente a $G' = \text{Convert}(G)$

CASO BASE: Se $K=2$ è banale. Per come abbiamo definito $\text{Convert}(G)$ si avrà che $G' = G$. Dunque $L(G') = L(G)$

PASSO INDUTTIVO: $\forall x \in L(G)$ si ha che $\exists$ un ramo di computazione in G che porta da q_{START} a q_{ACC} . Ora possiamo avere 2 casi:

- Il ramo di computazione NON passa per q_{rip} . Allora togliendo q_{rip} il ramo resta invariato $\Rightarrow L(G') = L(G)$
- Il ramo di computazione passa per q_{rip} . Allora, abbiamo una situazione tipo:



Ma, per come abbiamo definito S' , si ha che rimuovendo q_{rip} stiamo già considerando tutti i modi di andare da q_i a q_f passando per q_{rip} $\Rightarrow L(G') = L(G)$

$\Leftarrow$ L'idea, come prima, è quella di trasformare un espr. regolare in un NFA.
Per fare ciò possiamo usare la definizione (ricorsiva) di $L(vc)$ con $vc \in REGEX$.

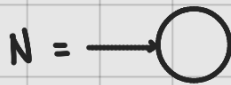
CASO BASE	$L(vc) = \begin{cases} \emptyset & \text{se } vc = \emptyset \\ \{\epsilon\} & \text{se } vc = \epsilon \\ \{a\} & \text{se } vc = a \end{cases}$
-----------	---

PASSO INDUTTIVO	$L(vc) = \begin{cases} L(R_1) \cup L(R_2) & \text{se } vc = R_1 \cup R_2 \\ L(R_1) \cdot L(R_2) & \text{se } vc = R_1 \cdot R_2 \\ L(R)^* & \text{se } vc = R^* \end{cases}$
-----------------	--

Dimostriamo per induzione, cioè che è possibile costruire un OFA/NFA per i casi base.
Il passo induttivo verrà automaticamente sfruttando le prop. di chiusura di REG per le op. di "u", "o" e "*".

CASO BASE:

$$L(v_e) = \emptyset$$



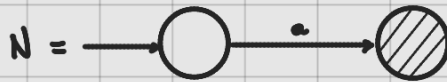
e si ha quindi $L(N) = \emptyset = L(v_e)$

$$L(v_e) = \{\varepsilon\}$$



e si ha quindi $L(N) = \{\varepsilon\} = L(v_e)$

$$L(v_e) = \{a\}$$



e si ha quindi $L(N) = \{a\} = L(v_e)$

PASSO INDUTTIVO:

$$L(v_e) = L(R_1) \cup L(R_2)$$

Per HP INDUTTIVA si ha che $\exists H_1, H_2 \in \text{NFA t.c.}$

$L(H_1) = L(R_1)$ e $L(H_2) = L(R_2)$. Utilizzando ora la prop. di chiusura di REG per "U" si ha che $\exists H \in \text{NFA t.c.}$ $L(H) = L(H_1) \cup L(H_2) = L(R_1) \cup L(R_2) = L(v_e)$

$$L(v_e) = L(R_1) \cdot L(R_2)$$

Per HP INDUTTIVA si ha che $\exists H_1, H_2 \in \text{NFA t.c.}$

$L(H_1) = L(R_1)$ e $L(H_2) = L(R_2)$. Utilizzando ora la prop. di chiusura di REG per "." si ha che $\exists H \in \text{NFA t.c.}$ $L(H) = L(H_1) \cdot L(H_2) = L(R_1) \cdot L(R_2) = L(v_e)$

$$L(v_e) = L(R)^*$$

Per HP INDUTTIVA si ha che $\exists H_1 \in \text{NFA t.c.}$ $L(H_1) = L(R)$

Utilizzando la prop. di chiusura di REG per "*" si ha che $\exists H \in \text{NFA t.c.}$

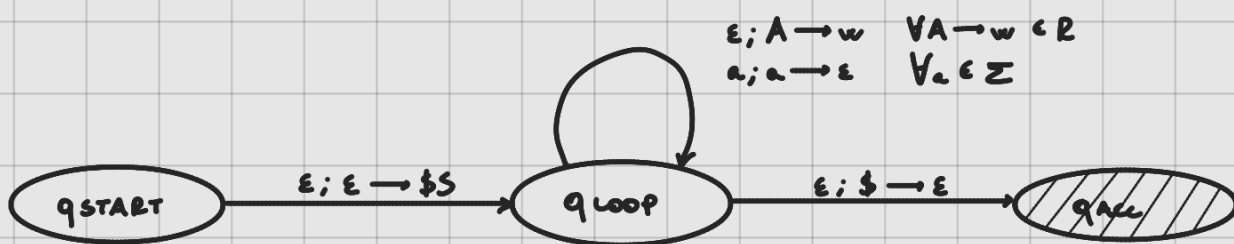
$$L(H) = L(H_1)^* = L(R)^* = L(v_e)$$

Dimostrare che un Linguaggio è Acontestuale sse. esiste un PDA che lo riconosce

② Dobbiamo mostrare che ogni grammatica può essere trasformata in un PDA e viceversa.

$$\Rightarrow \forall_{CFG \ G \ \exists_{PDA \ P} \ t.c. \ L(G) = L(P)$$

L'idea è quella di inserire le regole della grammatica G nello stack del PDA P . Ogni volta leggiamo la cima dello stack e se c'è una variabile la sostituiamo con la parte dx di una sua regola. Se c'è un terminale facciamo matching con la stringa in input. Se il matching fallisce (es. sto controllando una stringa che inizia con 'a' ma la grammatica genera stringhe che iniziano con 'b') allora P RIFIUTA. Se il matching ha successo rimuoviamo il terminale dalla cima e controlliamo il simbolo sottostante. Se ad un certo punto abbiamo pila vuota significa che abbiamo matchato con successo tutto l'input con le regole della grammatica $\Rightarrow P$ ACCETTA. Nella pratica avremo un PDA del tipo:



Formalmente $P = (Q, \Sigma, \Gamma, \delta, q_{start}, F)$ in cui:

- $Q = \{q_{start}, q_{loop}, q_{acc}\}$
- $\Sigma = \Sigma_G$
- $\Gamma = \Sigma_G \cup V_G$
- $F = \{q_{acc}\}$

e la δ è definita come:

- $\delta(q_{start}, \epsilon, \epsilon) = (q_{loop}, \$S)$

- $\delta(q_{loop}, \varepsilon, A) = (q_{loop}, w) \quad \forall A \rightarrow w \in R_q$
- $\delta(q_{loop}, a, a) = (q_{loop}, \varepsilon) \quad \forall a \in \Sigma$
- $\delta(q_{loop}, \varepsilon, \$) = (q_{acc}, \varepsilon)$

⇐ Dobbiamo trasformare un PDA P in una CFG G equivalente. A questo scopo consideriamo, invece, una variante del PDA P in cui:

- S_i ha un solo stato accettante
- Non si effettuano sostituzioni ma solo PUSH e POP
- Svuota lo stack prima di accettare

In questo modo avremo che $x \in L(P) \Leftrightarrow x$ porta P da q_{start} a q_{acc} con pila vuota. Creiamo quindi la nostra grammatica $G = (V, \Sigma, R, S)$ t.c.

- $V = \{A_{pq} : \forall p, q \in Q\}$ in cui A_{pq} = tutte le stringhe che portano il PDA P dallo stato p allo stato q con pila vuota
- $\Sigma = \Sigma_P$
- $S = A_{q_{start} q_{acc}}$

E regole:

$$① A_{pp} \rightarrow \varepsilon \quad \forall p \in Q$$

$$② A_{pq} \rightarrow a A_{rsb} \quad \text{se } (r, u) \in \delta(p, a, \varepsilon) \wedge (q, \varepsilon) \in \delta(s, b, u) \quad \begin{matrix} \forall p, q, r, s \in Q \\ \forall u \in \Gamma, a, b \in \Sigma \end{matrix}$$

$$③ A_{pq} \rightarrow A_{pr} A_{rq} \quad \forall p, q, r \in Q$$

Dimostriamo per induzione che G genera la stringa $x \Leftrightarrow x$ porta P dallo stato q_{start} allo stato q_{acc}

$\Rightarrow$ G genera la stringa $x \Rightarrow x$ porta P dallo stato q_{start} allo stato q_{acc}

HP INDUTTIVA: $\forall K \geq 1$ se $A_{pq} \xRightarrow{*} x$ con un numero di passi $\leq K$ allora x porta P dallo stato p allo stato q

CASO BASE: Se $K=1$ l'unica regola possibile per derivare x è $A_{pp} \rightarrow \epsilon$. Che ovviamente porta P dallo stato p allo stato p con pila vuota.

PASSO INDUTTIVO: Assumiamo che $A_{pq} \xRightarrow{*} x$ in $K+1$ passi. A questo punto:

- ① Se la prima regola applicata è $A_{pq} \rightarrow aAr_s b$ si ha che $x = ayb$ con $A_{rs} \xRightarrow{*} y$.
Ma, per HP INDUTTIVA, si ha che A_{rs} porta P dallo stato r allo stato s con pila vuota. Inoltre, per come abbiamo definito la regola ①, questa viene inserita solo se $(r, u) \in \delta(p, a, \epsilon) \wedge (q, \epsilon) \in \delta(s, b, u)$. Dunque il carattere u che ho inserito in p viene rimosso in $q \Rightarrow$ Porta P da p a q con pila vuota.
- ② Se la prima regola applicata è $A_{pq} \rightarrow A_{pr} A_{rq}$ si ha che $x = yz$ con $A_{pr} \xRightarrow{*} y$ e $A_{rq} \xRightarrow{*} z$. Ma, per HP INDUTTIVA si avrà che P va dallo stato $p \rightarrow r$ e dallo stato $r \rightarrow q$ con pila vuota $\Rightarrow P$ va dallo stato p a q con pila vuota.

$\boxed{\Leftarrow}$ x porta P dallo stato q_{start} allo stato $q_{acc} \Rightarrow G$ genera la stringa x

HP INDUTTIVA: $\forall K \geq 1$ se x porta P dallo stato p allo stato q in K passi allora $A_{pq} \stackrel{*}{\Rightarrow} x$

CASO BASE: Se $K=0$ significa che l'unica stringa possibile è $x=\epsilon$. La grammatica G contiene la regola $A_{pp} \rightarrow \epsilon$.

PASSO INDUTTIVO: Supponiamo che P vada da p a q con pila vuota in $K+1$ passi. Dobbiamo mostrare che $A_{pq} \stackrel{*}{\Rightarrow} x$. Abbiamo 2 casi:

① La pila non si svuota nel mezzo. Dunque alla prima mossa P fa push di $u \in \Gamma$ e nell'ultima mossa fa pop di quello stesso u . Siano $a, b \in \Sigma$ rispettivamente i caratteri letti da P alla prima e all'ultima mossa ed $r, s \in Q$ gli stati rispettivamente dopo la prima mossa e dopo l'ultima. Si avrà che $A_{pq} \stackrel{*}{\Rightarrow} x = aA_{rs}b$

Ma, poiché abbiamo già analizzato il passo iniziale e finale di A_{pq} , si ha che A_{rs} richiede $K+1-2 = K-1$ passi. Possiamo quindi applicare l'HP INDUTTIVA e concludere che $A_{rs} \stackrel{*}{\Rightarrow} y$. E dunque nel complesso $A_{pq} \stackrel{*}{\Rightarrow} x = ayb$.

② La pila si svuota nel mezzo. Significa che nello stato p facciamo push di un certo $u \in \Gamma$, che verrà estratto in un certo stato r . Sia $x = yz$. Poiché la computazione da $p \rightarrow r$ e da $r \rightarrow q$ richiede entrambe un numero di passi $\leq K$ possiamo usare l'HP INDUTTIVA e concludere che $A_{pr} \stackrel{*}{\Rightarrow} y$ e $A_{rq} \stackrel{*}{\Rightarrow} z$. Dunque $A_{pq} \stackrel{*}{\Rightarrow} x = yz$.

Enunciare e dimostrare il Pumping Lemma per i Linguaggi Regolari. Fornire un esempio di utilizzo

Teorema: Sia $L \in \text{Reg}$ allora $\exists p \in \mathbb{N}$ t.c. $\forall w \in L$, con $|w| \geq p$, w può essere scomposta in $w = xyz$ dove:

- $|xy| \leq p$
- $|y| > 0$
- $\forall i \geq 0 \quad xy^iz \in L$

Dim. Sia $L \in \text{Reg}$ ed $M = (Q, \Sigma, \delta, q_0, F)$ t.c. $L(M) = L$. Sia $p = |Q|$, $w = w_1 \dots w_n \in \Sigma^*$ con $|w| \geq p$ e siano $r_1, \dots, r_{n+1}$ gli stati attraversati da $M(w)$.

Siccome $|w| \geq p$ si ha che nella sequenza di stati attraversati $r_1, \dots, r_{n+1}$ almeno uno stato si ripete. Sia r_j la prima occorrenza del primo stato che si ripete, e sia r_ℓ la seconda. Sicuramente vale $j < \ell$. Inoltre, possiamo dividere $w = xyz$ t.c.

- $x = w_1 \dots w_{j-1}$
- $y = w_j \dots w_{\ell-1}$
- $z = w_\ell \dots w_n$

Avremo che:

- x porta M da r_1 ad r_j
- y porta M da r_j ad r_ℓ
- z porta M da r_ℓ ad r_{n+1}

Dunque, $\forall i \geq 0$ si avrà che $xy^iz \in L$. Inoltre:

- siccome $y = w_j \dots w_{\ell-1}$ e $j < \ell$, si avrà sempre $|y| > 0$
- siccome $xy = w_1 \dots w_{\ell-1}$ e $\ell \leq p$, si avrà sempre $|xy| \leq p$

Esempio: $L = \{0^n 1^n : n \geq 0\}$ non è regolare

Applicando pumping lemma ci basta prendere $w = 0^p 1^p$. Chiaramente $|w| = 2p \geq p$. Dunque possiamo scomporre $w = xyz$. In particolare, siccome $|xy| \leq p$ si avrà sicuramente che y contiene solo '0'.

Dunque facendo sia pumping up ($i > 1$) che pumping down ($i = 0$) andremmo ad alterare il numero di '0' mentre il numero di '1' rimane fermo $\Rightarrow xy^i z \notin L$.

Dimostrare che un Linguaggio è Regolare sse. esiste una NFA che lo riconosce

Nella pratica dobbiamo dimostrare che $L(\text{DFA}) = L(\text{NFA})$. Procediamo con la doppia inclusione

$L(\text{DFA}) \subseteq L(\text{NFA})$ Se $L \in L(\text{DFA})$ allora $\exists D \in \text{DFA}$ t.c. $L(D) = L$.

Ma, siccome ogni DFA è anche un NFA, basta prendere $N = D$ ed avremo che $\exists N \in \text{NFA}$ t.c. $L(N) = L$. Dunque $L \in L(\text{NFA})$

$L(\text{NFA}) \subseteq L(\text{DFA})$ Se $L \in L(\text{NFA})$ allora $\exists N \in \text{NFA}$ t.c. $N = (Q_N, \Sigma, \delta_N, q_0^N, F_N)$ con $L(N) = L$. Dobbiamo quindi costruire un DFA $D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$ t.c. $L(D) = L$.

- $Q_D = P(Q_N)$
- $q_0^D = E(\{q_0^N\})$
- $F_D = \{R \in Q_D : R \cap F \neq \emptyset\}$
- $\delta_D(R, a) = \bigcup_{r \in R} E(\delta_N(r, a)) \quad \forall R \in Q_D ; a \in \Sigma$

dove $E(R) = \{q \in Q_N : \exists r \in R \text{ t.c. } \delta_N(r, \varepsilon) = q\}$

Definire le Grammatiche Acontestuali e dimostrare che ogni grammatica acontestuale ne ammette una equivalente in Forma Normale di Chomsky

Una CFG - Context Free Grammar G è una tupla (V, Σ, R, S) in cui:

- V è l'insieme delle Variabili
- Σ è l'insieme dei terminali (anche detto alfabeto)
- R è l'insieme delle Regole
- S è la variabile iniziale

Ogni CFG G ammette una CFG G' in forma normale di Chomsky equivalente, ovvero $L(G) = L(G')$. Una CFG contiene tutte regole nella forma:

- $A \rightarrow BC$
- $A \rightarrow a$
- $S \rightarrow \varepsilon$

con $A \in V$; $B, C \in V \setminus \{S\}$ ed $a \in \Sigma \setminus \{\varepsilon\}$.

Per dimostrare che $L(G) = L(G')$ basta mostrare che ogni grammatica può essere trasformata in forma normale e viceversa.

$\Rightarrow$ Data una qualsiasi CFG G possiamo trasformarla in una CFG G' in forma normale di Chomsky seguendo questo algoritmo:

- 1) Fare in modo che la variabile iniziale S sia solo nel lato sx delle regole.
Per fare ciò basta aggiungere una nuova variabile iniziale S' ed una regola $S' \rightarrow S$. Questo è vero perché, per transitività S' potrà generare tutte le stringhe generate da S .

2) Rimuovere tutte le ϵ -regole. L'unica ammessa è $S' \rightarrow \epsilon$. Per rimuovere le ϵ -regole basta togliere $A \rightarrow \epsilon$ dalle regole e adattare tutte le altre regole al cambiamento. Ad esempio con $A \rightarrow \epsilon$ e $B \rightarrow ACD$ otterremo la regola $B \rightarrow \epsilon CD = CD$. Anche qui stiamo solo applicando transitività $\Rightarrow$ il Linguaggio rimane invariato.

3) Fare in modo che tutte le regole abbiano esattamente 2 variabili o 1 terminale e ϵ .

- Rimuovere le regole unitarie $A \rightarrow B$ sostituendo a B il valore (parte ϵ) delle sue regole. Transitività $\Rightarrow$ Linguaggio invariato.
- Se abbiamo una regola $A \rightarrow Ba \in R_G$ creiamo una nuova variabile A' ed sostituiamo $A \rightarrow Ba$ con $A \rightarrow BA'$ dove $A' \rightarrow a \in R$. Anche qui transitività $\Rightarrow$ Linguaggio invariato
- Se abbiamo una regola $A \rightarrow BCD \in R_G$ creiamo una nuova variabile A' ed sostituiamo $A \rightarrow BCD$ con $A \rightarrow BA'$ dove $A' \rightarrow CD \in R$. Anche qui transitività $\Rightarrow$ Linguaggio invariato

Poiché nessuno di questi step (1, 2, 3, 4) modifica il linguaggio si ha che $L(G) = L(G')$

$\boxed{\Leftarrow}$ Una grammatica accontestuale in forma normale di Chomsky è banalmente una grammatica accontestuale

Mostrare che in una grammatica in Forma Normale di Chomsky ogni derivazione di una stringa w tc. $|x| = n$ richiede al più $2n - 1$ passi (per ogni $n > 1$)

HP INDUTTIVA: Ogni derivazione di una stringa w lunga k richiede un numero di passi $\leq 2k - 1$

CASO BASE: Se $k=1$ si avrà che w è composta da un singolo carattere $a \in \Sigma$. Ma, l'unico modo di ottenere un singolo terminale in una grammatica in forma normale è utilizzando la regola $S \rightarrow a$. Abbiamo quindi $\#passi = 2(1) - 1 = 1 \Rightarrow$ CORRETTO.

PASSO INDUTTIVO: Consideriamo ora una stringa di lunghezza $k > 1$. Poiché la nostra grammatica è in forma normale si ha necessariamente che è stata utilizzata la regola $S \rightarrow AB$, in cui ogni variabile genera una sua sottostringa ($A \xRightarrow{*} w_1$ e $B \xRightarrow{*} w_2$)

Siamo quindi in una situazione tipo $w = w_1 w_2$ in cui $|w_1| = k_1$, $|w_2| = k_2$ e $|w| = k_1 + k_2 = k$. Ora, poiché sia $k_1 < k$ che $k_2 < k$, possiamo applicare l'HP INDUTTIVA e affermare che:

- $\#passi\ w_1 \leq 2k_1 - 1$
- $\#passi\ w_2 \leq 2k_2 - 1$

E siccome $w = w_1 w_2$ si avrà che:

$$\begin{aligned}\#passi\ w &= \#passi\ w_1 + \#passi\ w_2 + \overset{\substack{\text{il primo passo} \\ \uparrow}}{1} \quad S \rightarrow AB \\ &= 2k_1 - 1 + 2k_2 - 1 + 1 = \\ &= 2(k_1 + k_2) - 2 + 1 = \\ &= 2k - 1\end{aligned}$$

Definire la nozione di Push-Down Automaton (PDA)

Un PDA è una tupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove Q, Σ, q_0, F sono uguali ai DFA/NFA, mentre:

- Γ è l'alfabeto della PILA
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \longrightarrow P(Q \times \Gamma_\epsilon)$

Sostanzialmente un PDA è un NFA con una PILA LIFO. Ad ogni passo di computazione il PDA può operare sulla cima della PILA tramite:

- PUSH: inserisce un nuovo simbolo in cima
- POP: rimuove il simbolo in cima
- SOSTITUZIONE: cambia il simbolo in cima

L'alfabeto di Pila $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$ può essere diverso dall'alfabeto dell'automa Σ_ϵ